ПРАКТИЧЕСКАЯ РАБОТА «СЕТЕВЫЕ ЗАПРОСЫ И АСИНХРОННОСТЬ. FETCH, PROMISES, РАБОТА С API.»

***Цель**: научиться получать данные из интернета (fetch), обрабатывать ошибки сети и сервера, визуализировать результаты, управлять UX-состояниями (loading / error / success), подготовить фундамент для работы с сетевыми запросами в будущих SPA-проектах (курсовых).*

Подготовка проекта (как в реальной работе)

1. Создайте папку проекта: `js-practice-08`
2. Откройте папку в VS Code.
3. Чтобы не тратить время на CSS, используйте готовую верстку из файла `index.html` и оформление из файла `style.css`. Основная работа выполняется в файле: `script.js`.
4. Создайте репозиторий GitHub с названием `js-practice-08` и загрузите туда проект.

Структура репозитория

```
js-practice-08/
├── index.html
├── style.css          ← изменяем по мере надобности
├── script.js          ← реализуем
├── README.md          ← ответы на вопросы для интервью
└── screenshots/
    ├── 01_initial.png
    ├── 02_loading.png
    ├── 03_result.png
    ├── 04_error.png
    └── 05_console.png
```

В **README.md** кратко опишите:

- Название проекта
- Используемые технологии (HTML, CSS, JS)
- Ответы на контрольные вопросы (смотри соответствующий раздел)

Введение

Вы уже знаете основы JS, DOM и событий. Сейчас важно научиться получать данные из внешних API, правильно обрабатывать ошибки и состояния загрузки — это ключевой навык для реальных SPA.

Общие требования

☑ Обязательно:

✓ `async/await` — основной синтаксис;

✓ `try...catch` — обработка ошибок (не оставляйте пустые catch);

✓ всегда проверяйте `response.ok` перед `response.json()`;

✓ комментарии «ПОЧЕМУ?» рядом с каждым асинхронным вызовом;

✓ loader (spinner или текст) виден во время запросов;

✓ не используйте `innerHTML` для данных от API — создавайте элементы через `createElement` и вставляйте безопасно.

🚫 ТАБУ (нарушение = минус балл + «удар» палкой по спине от бабушки-айтишницы):

✗ запросы без обработки ошибок;

✗ синхронный код в местах, где нужен асинхронный;

✗ `innerHTML` для вставки данных от API.

Концепция проекта: GitHub Spy — Поиск разработчиков

Мы создадим приложение, которое ищет профили пользователей на GitHub и выводит их публичные репозитории.

UI-предметы

- Поле ввода (`input#username`), кнопка поиска (`button#searchBtn`);
- Блок ошибок (`div#error`);
- Loader (элемент `.loader`);
- Контейнер для карточки профиля (`div#profile`);
- Контейнер для списка репозиториев (`ul#repos`);
- История поиска (`div#history`);
- Кнопка очистки истории (`button#clearHistory`).

API и эндпоинты

- Профиль: https://api.github.com/users/USERNAME
- Репозитории: https://api.github.com/users/USERNAME/repos?sort=updated&per_page=5

Пошаговое задание

Шаг 1 — БАЗА (минимум для зачёта, ~2–3 ч)

1.1 Реализуйте async function getUser(username)

- fetch к эндпоинту профиля.
- Если !response.ok:
  - Если response.status === 404 — покажите сообщение «Пользователь не найден».
  - Иначе — покажите общую ошибку с кодом статуса.
- ПОЧЕМУ? комментарий: почему проверяем response.ok.

1.2 Отображение профиля

- При клике по кнопке «Найти» получите username из input, запустите getUser, отрисуйте карточку профиля:
  - img (avatar_url), h2 (name или login), p (bio), span (public_repos).
- Используйте createElement и textContent.

Шаг 2 — JUNIOR (основной уровень, ~3 ч)

2.1 Цепочка запросов

- После успешного профиля выполните fetch репозиториев, отрисуйте список последних 5 репозиториев (name — ссылка, description, stargazers_count).
- ПОЧЕМУ? комментарий: зачем делать второй запрос после профиля.

2.2 Обработка состояний (UX)

- Loader: показывайте перед отправкой запросов и скрывайте после завершения (в finally).
- Блокируйте кнопку (disabled = true) во время запроса.
- Показывайте читаемое сообщение об ошибке в div#error (красным).

2.3 README (собеседование)

В README.md ответьте на вопросы:

1. Чем async/await лучше чистых .then()?
2. Почему fetch не падает в catch при ошибке 404?
3. Зачем проверять Content-Type ответа?

🔴 Шаг 3 — PRO-расширения (опционально, +25%)

3.1 История поиска в localStorage

- Сохраняйте последние 3 успешных username в localStorage (например, under key 'gh_search_history').
- При загрузке страницы отрисовывайте их как кликабельные теги; клик по тегу выполняет новый поиск.

3.2 Promise.all (параллельные запросы)

- Вместо последовательных запросов запустите оба: профиль и репозитории через Promise.all (или Promise.allSettled), обработайте результаты.
- ПОЧЕМУ? комментарий: преимущества параллельных запросов для ускорения.

🟢 Бонусные задания (для прогрессии)

- Добавьте debounce на input (300ms).
- Реализуйте пагинацию для репозиториев.
- Сохранение предпочитаемой единицы форматирования (например, количество репов) в localStorage.
- Добавьте unit-тесты для утилитарных функций (например, normalizeUsername).

Автопроверка — QuickCheck (вставить в консоль)

Скопируйте код ниже, вставьте в DevTools → Console при открытой странице и нажмите Enter:

```
(function checkAPI(){
const hasAsync = /async\s+function|await\s+fetch/i.test(document.scripts[0]?.innerText || "");
const hasTryCatch = /try\s*{|catch\s*(/.test(document.scripts[0]?.innerText || "");
console.table([
{ критерий: "Использование async/await", статус: hasAsync ? "☑" : "✗" },
{ критерий: "Обработка ошибок try/catch", статус: hasTryCatch ? "☑" : "✗" },
{ критерий: "Наличие Loader (элемента)", статус: !!document.querySelector('.loader') ? "☑" : "⚠ (проверь селектор)" }
]);
})();
```

Шаблоны комментариев «ПОЧЕМУ?»

Скопируйте и вставьте рядом с соответствующим кодом:

```
// ПОЧЕМУ async/await? —  — Позволяет писать асинхронный код так, будто он синхронный; читается легче, чем цепочки .then().
// ПОЧЕМУ !response.ok? - fetch не считает ошибки 404/500 исключениями; нужно вручную проверять успешность статус-кода.
// ПОЧЕМУ try...catch? —  Единственный способ поймать ошибки сети (например, пропал интернет) при использовании await.
```

Требования к скриншотам

1. `01_initial.png` — Пустая страница поиска.
2. `02_loading.png` — Момент запроса (виден индикатор загрузки).
3. `03_result.png` — Успешно найденный профиль и список репозиториев.
4. `04_error.png` — Сообщение «Пользователь не найден».
5. `05_console.png` — Вкладка Network с успешным запросом (status 200).

Советы и распространенные ошибки и как их избежать

- GitHub Rate Limit: если видите 403/Rate Limit, подождите ~15 минут или переключитесь на мобильный интернет.
- Всегда проверяйте response.headers.get('Content-Type') перед парсингом JSON.
- Блокируйте кнопку поиска, чтобы избежать множественных параллельных кликов.
- Не оставляйте пустые catch — всегда логируйте ошибку и показывайте пользователю понятное сообщение;
- Для гуманитариев: начните с БАЗЫ, шаг за шагом; копируйте примерные структуры, читайте ошибки в консоли.
- Для IT-специалистов: добавьте обработку edge-case'ов (rate limit, пустые поля bios, приватные репы), используйте Promise.all и debouncing для input.
- Для желающих на джуниор-позиицию: подготовьте README с ответами на вопросы и небольшим рефакторингом кода (модули, функции, комментарии «ПОЧЕМУ?») — это пригодится на собеседовании.

Вопросы для интервью — ответы для README.md

Напишите ответы в README.md своего репозитория.

1. **Чем async/await лучше чистых .then()?**
2. **Почему fetch не падает в catch при ошибке 404?**
3. **Зачем проверять Content-Type ответа?**

---

9. Чек-лист сдачи и критерии оценки

- Нет «Callback Hell» (каскад .then()).
- Использованы async/await (в основном задании) и .then() (в разминке).
- 404 аккуратно обрабатывается.

- Кнопка поиска выключается во время загрузки.
- (PRO) История поиска хранится в localStorage (последние 3).

- (PRO) Promise.all использован для параллельных запросов (при желании).

Критерий	Уровень	Баллы
Выполнено	🟢 БАЗА	60%
Выполнено	🟡 JUNIOR	+30%
Выполнено (history + Promise.all)	🔴 PRO	100%
Чистота кода, комментарии «ПОЧЕМУ?», README и скриншоты учитываются дополнительно.		

Форма сдачи: Отправьте преподавателю ссылку на GitHub-репозиторий в Slack

Примечание: при необходимости вы можете упростить стили. Оценивается логика, корректность условий, обработка ошибок и аккуратность кода.

Удачи! Если застряли — перечитайте раздел «Частые ошибки» и запустите QuickCheck.